

Design Choices in Neural Networks

DeNederlandscheBank

EUROSYSTEEM



DataScience
Hub

Michiel Nijhuis
Iman van Lelyveld

Content

1. Design choices in neural networks

- Feature engineering vs. learning representations with many parameters
- Backpropagation
- Optimizers

2. Training deep learning models

- Minibatch stochastic gradient descent
- Vanishing/exploding gradients and the role of regularization layers

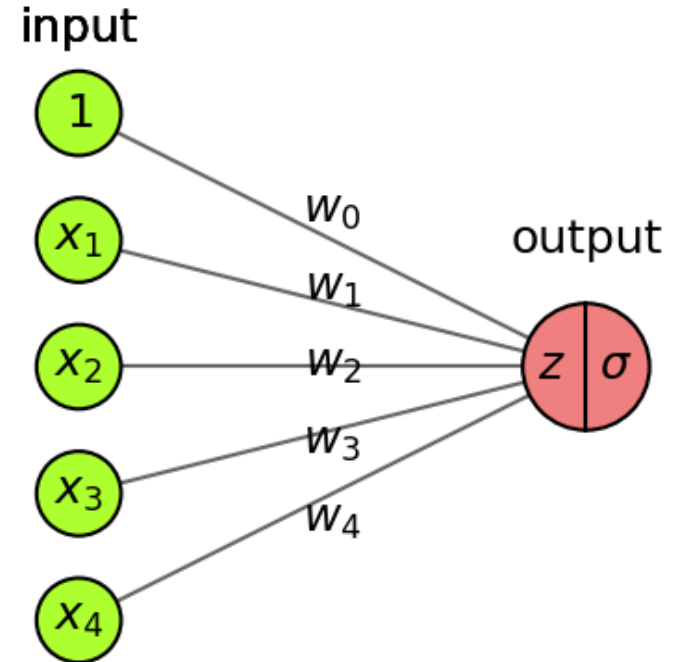
3. Architecture

- Convolutional neural networks

From logistic regression

- Logistic regression, drawn in a different, neuro-inspired, way
 - Linear model: inner product z of input vector x and weight vector w , plus bias w_0
 - Logistic (or sigmoid) function maps the output to a probability in $[0,1]$
 - Uses log loss (cross-entropy) and gradient descent to learn the weights

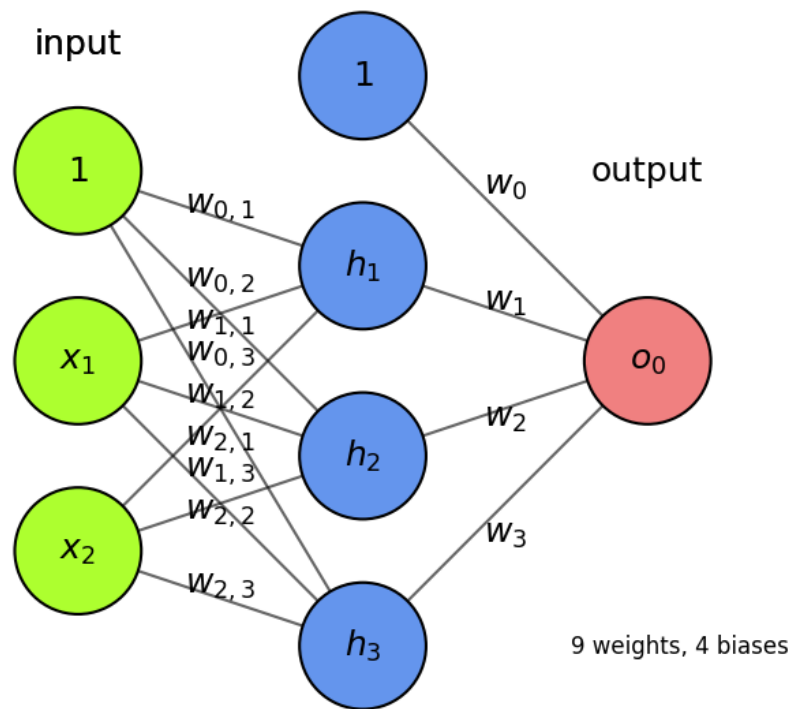
$$\begin{aligned}\hat{y}(x) &= \text{sigmoid}(z) \\ &= \text{sigmoid}(w_0 + w_x) \\ &= \text{sigmoid}(w_0 + w_1 * x_1 + w_2 * x_2 + \dots + w_p * x_p)\end{aligned}$$



To neural networks

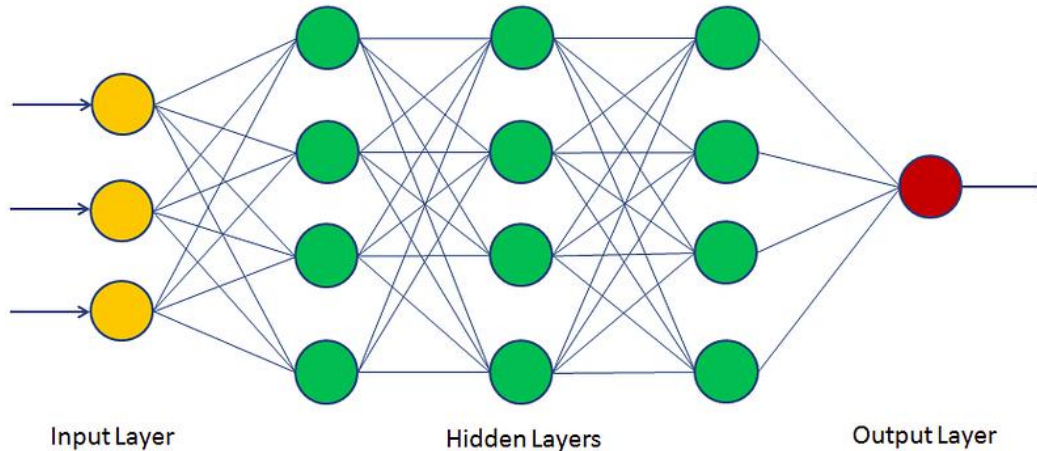
- Add one (or more) *hidden* layers h with k nodes (or units, cells, neurons)
 - Every 'neuron' is a tiny function, the network is an arbitrarily complex function, in the simplest form it weighs, sums the inputs and applies an activation function
 - Weights $w_{i,j}$ between node i and node j form a weight matrix W_l per layer l

$$\hat{y}(x) = \text{sigmoid}(w_0 + w_1 * \text{sigmoid}(w_{0,1} + w_{1,1} * x_1 + w_{2,1} * x_2) + w_2 * \text{sigmoid}(w_{0,2} + w_{1,2} * x_1 + w_{2,2} * x_2) + w_3 * \text{sigmoid}(w_{0,3} + w_{1,3} * x_1 + w_{2,3} * x_2))$$



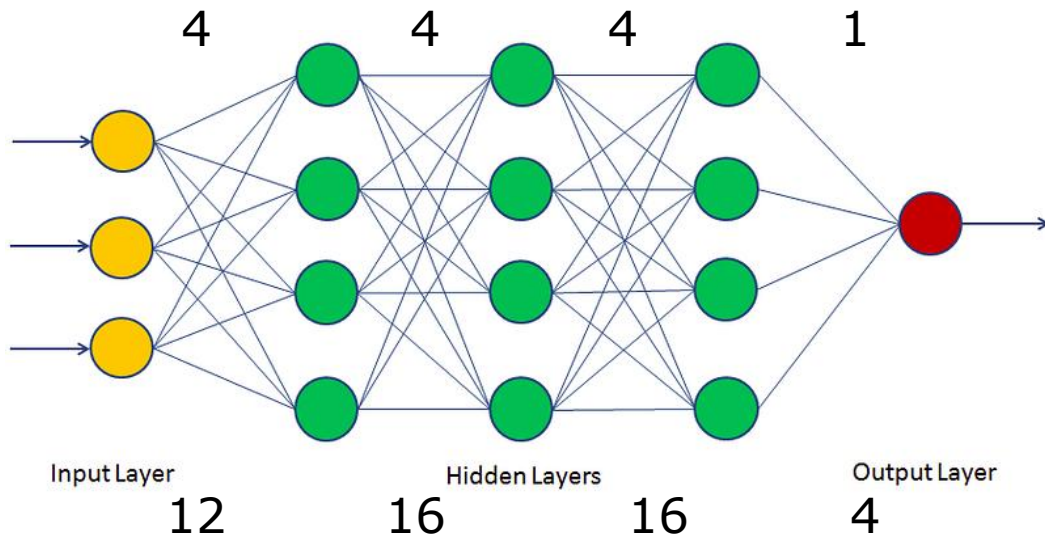
Adding more layers

- Add more layers, and more nodes per layer, to make the model more complex
 - For simplicity, we don't draw the biases (but remember that they are there)
- In *dense* (fully-connected) layers, every previous layer node is connected to all nodes
- The output layer can also have multiple nodes (e.g. 1 per class in multi-class classification)



Adding more layers

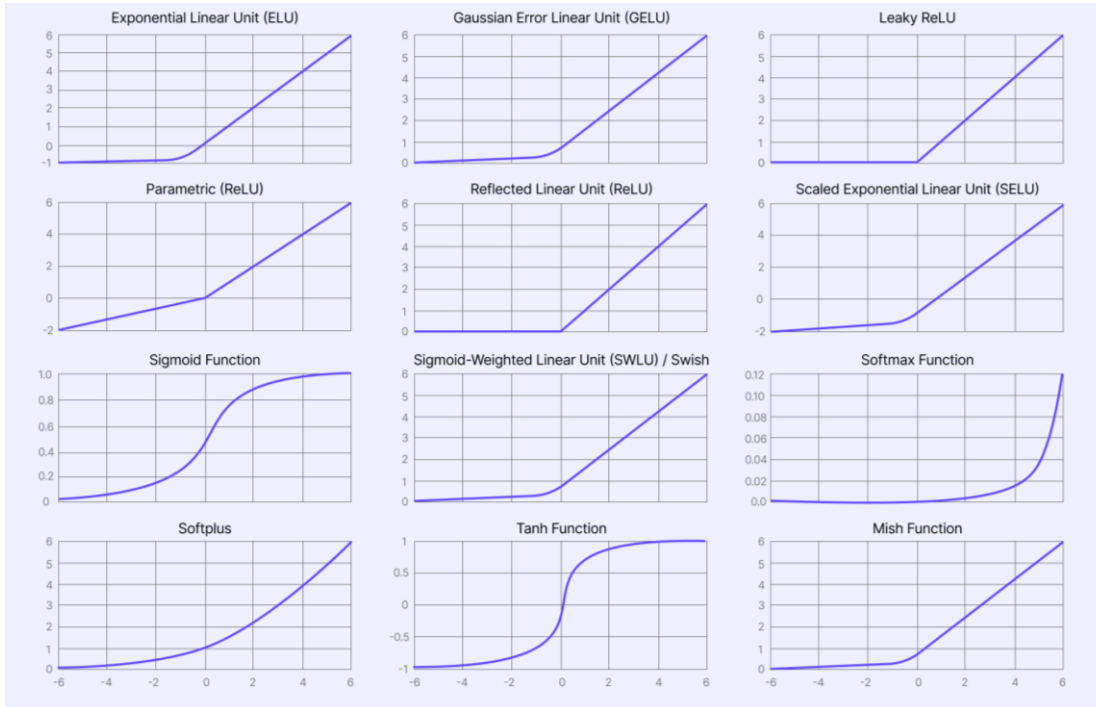
- Add more layers, and more nodes per layer, to make the model more complex
 - For simplicity, we don't draw the biases (but remember that they are there)
- In *dense* (fully-connected) layers, every previous layer node is connected to all nodes
- The output layer can also have multiple nodes (e.g. 1 per class in multi-class classification)



Feature engineering vs learning representations

<https://playground.tensorflow.org/>

Activation functions

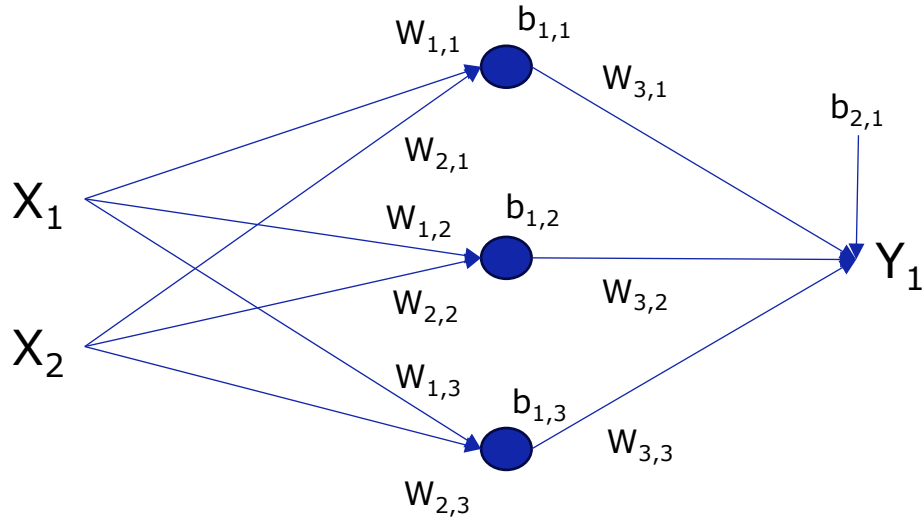


Preprocessing

Preprocessing: Normalisation, Reshaping, Encoding

- Always normalise (standardise or min-max) the inputs. Mean should be close to 0.
 - Avoid that some inputs overpower others
 - Speed up convergence
 - Gradients of activation functions are (near) 0 for large inputs
 - If some gradients become much larger than others, SGD will start zig-zagging
- Reshape the data to fit the shape of the input layer, e.g. (n, 16*16) or (n, 16,16)
 - Tensor with instances in first dimension, rest must match the input layer
- In multi-class classification, every class is an output node, so one-hot-encode the labels
 - e.g. class '4' becomes [0,0,0,0,1,0,0,0,0,0]

Neural network

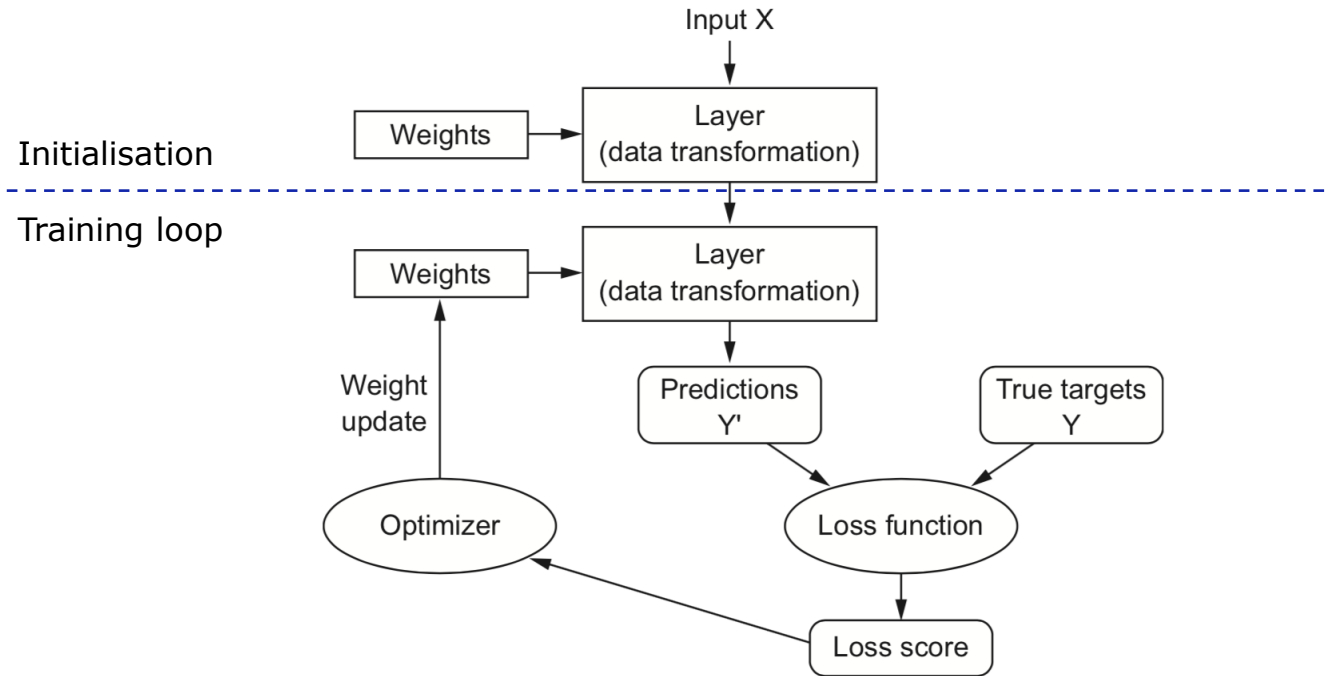


$$f(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

$$Y_1 = \frac{1}{1 + e^{-x}}$$

$$Y_1 = \frac{1}{1 + e^{-\left(b_{2,1} - w_{3,1} \cdot \max(0, b_{1,1} + w_{1,1}x_1 + w_{2,1}x_2) - w_{3,2} \cdot \max(0, b_{1,2} + w_{1,2}x_1 + w_{2,2}x_2) - w_{3,3} \cdot \max(0, b_{1,3} + w_{1,3}x_1 + w_{2,3}x_2)\right)}}$$

Training neural networks



Training algorithms

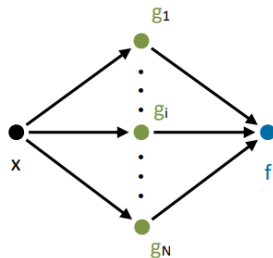
1. Take training data X and y
2. Forward pass: pass X through the network to yield predictions $\hat{y}$
3. Compute the loss L (mismatch between $\hat{y}$ and y)
4. Backward pass: Compute the gradient of the loss with regard to every weight
 - *Backpropagate* the gradients through all the layers
5. Update

$$W : W_{(i+1)} = W_{(i)} - \frac{\partial L(x, W_{(i)})}{\partial W} * \eta$$

Repeat until n passes (epochs) are made through the entire training set

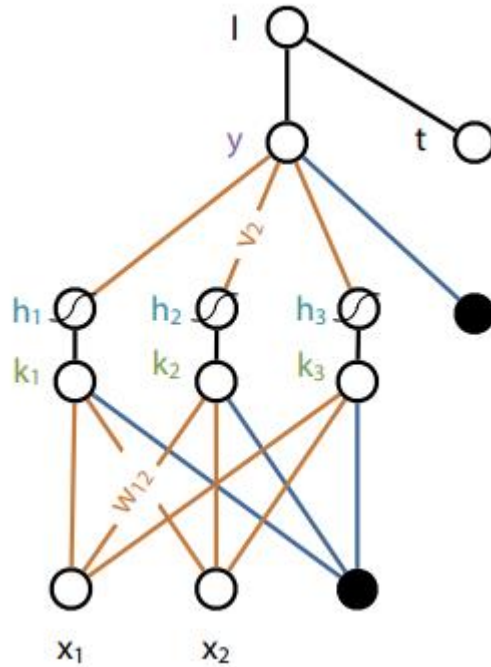
Backpropagation


$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial x}$$

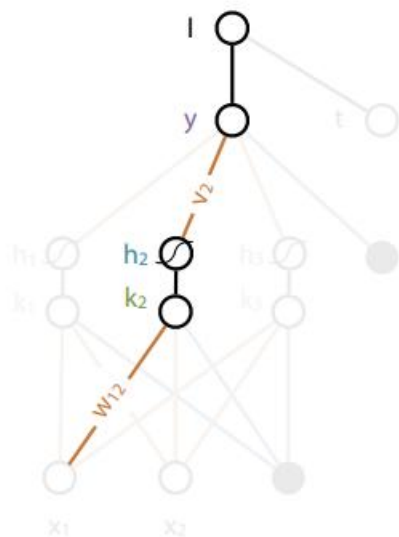


$$\frac{\partial f}{\partial x} = \sum_i \frac{\partial f}{\partial g_i} \frac{\partial g_i}{\partial x}$$

Backpropagation



Backpropagation



$$\frac{\partial l}{\partial y}$$

$$\frac{\partial l}{\partial y} \frac{\partial y}{\partial h_2} = \frac{\partial l}{\partial h_2}$$

$$\frac{\partial l}{\partial y} \frac{\partial y}{\partial h_2} \frac{\partial h_2}{\partial k_2} = \frac{\partial l}{\partial k_2}$$

$$\frac{\partial l}{\partial y} \frac{\partial y}{\partial h_2} \frac{\partial h_2}{\partial k_2} \frac{\partial k_2}{\partial w_{12}} = \frac{\partial l}{\partial w_{12}}$$

Optimisers

Momentum

- Imagine a ball rolling downhill: accumulates momentum, doesn't exactly follow steepest descent
 - Reduces oscillation, follows larger (consistent) gradient of the loss surface
- Adds a velocity vector
- Nesterov momentum: Look where momentum step would bring you, compute gradient there
 - Responds faster (and reduces momentum) when the gradient changes

Adaptive gradients

- 'Correct' the learning rate for each weight based on specific local conditions (layer depth, fan-in,...)
- Adagrad: scale η according to squared sum of previous gradients
 - RMSProp: use moving average of squared gradients

Adam (Adaptive moment estimation)

- Adam: RMSProp + momentum.
- Adds moving average for gradients as well
- Adds a bias correction to avoid small initial gradients

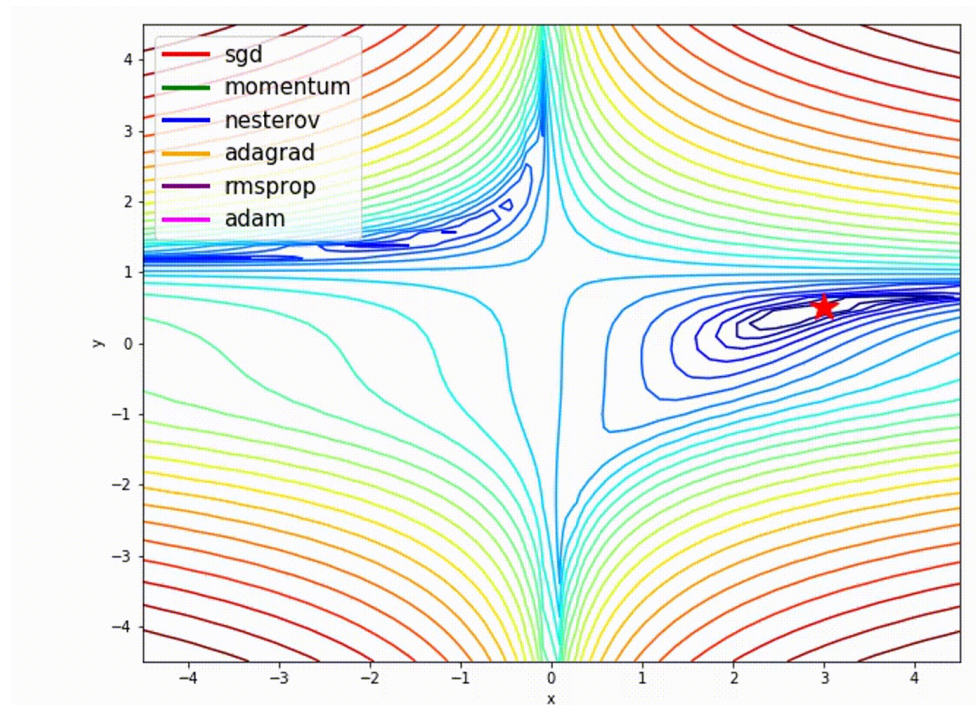
Training algorithms

Mini-batch Stochastic Gradient Descent

1. Draw a batch from the training data X and y
2. Forward pass: pass X through the network to yield predictions $\hat{y}$
3. Compute the loss L (mismatch between $\hat{y}$ and y)
4. Backward pass: Compute the gradient of the loss with regard to every weight
 - *Backpropagate* the gradients through all the layers
5. Update $W : W_{(i+1)} = W_{(i)} - \frac{\partial L(x, W_{(i)})}{\partial W} * \eta$

Repeat until n passes (epochs) are made through the entire training set

Performance



Performance

Choosing loss & optimizer

○ Loss function

- Use softmax to get to the labels

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

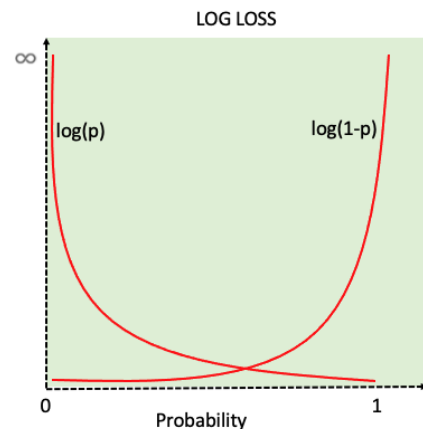
- Cross-entropy (log loss) for multi-class classification $\hat{y}$ is one-hot encoded)

$$L = - \sum_i y_i \log(\hat{y}_i)$$

- Use binary cross-entropy for binary problems (single output node)

• Optimizer

- Any of the optimizers we discussed before. Adam usually works well.



Hyperparameters

Choosing training hyperparameters

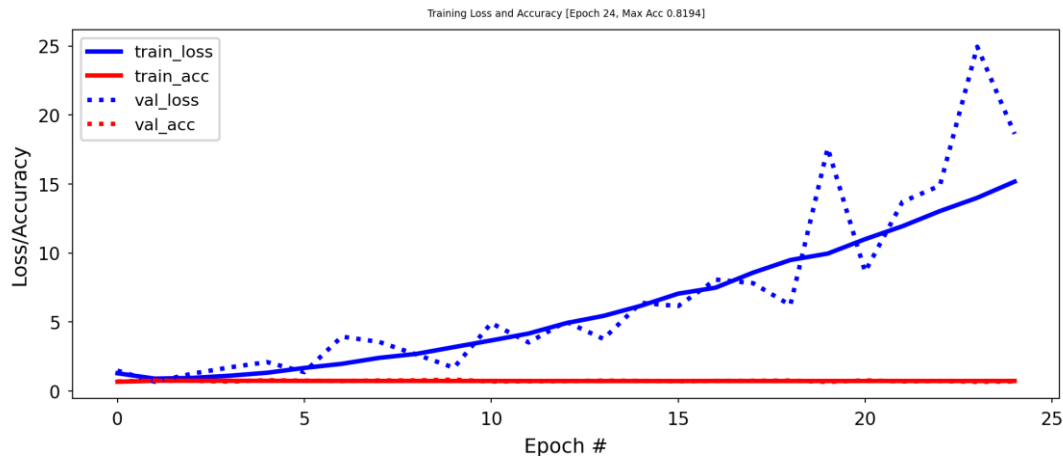
- Number of epochs: enough to allow convergence
 - Too much: model starts overfitting (or just wastes time)
- Batch size: small batches (e.g. 32, 64,... samples) often preferred
 - 'Noisy' training data makes overfitting less likely
 - Larger batches generalize less well ('generalization gap')
 - Requires less memory (especially in GPUs)
 - Large batches do speed up training, may converge in fewer epochs
- Batch size interacts with learning rate
 - Instead of shrinking the learning rate you can increase batch size

How to know the model is trained well

- How many epochs do we need for training?
- Train the neural net and track the loss after every iteration on a validation set
 - You can add a callback to the fit version to get info on every epoch

Early stopping

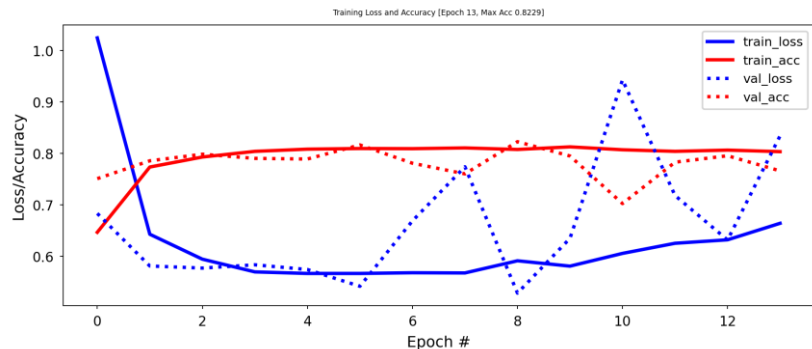
- Stop training when the validation loss (or validation accuracy) no longer improves
- Loss can be bumpy: use a moving average or wait for k steps without improvement



How to know the model is trained well

Regularisation and memorisation capacity

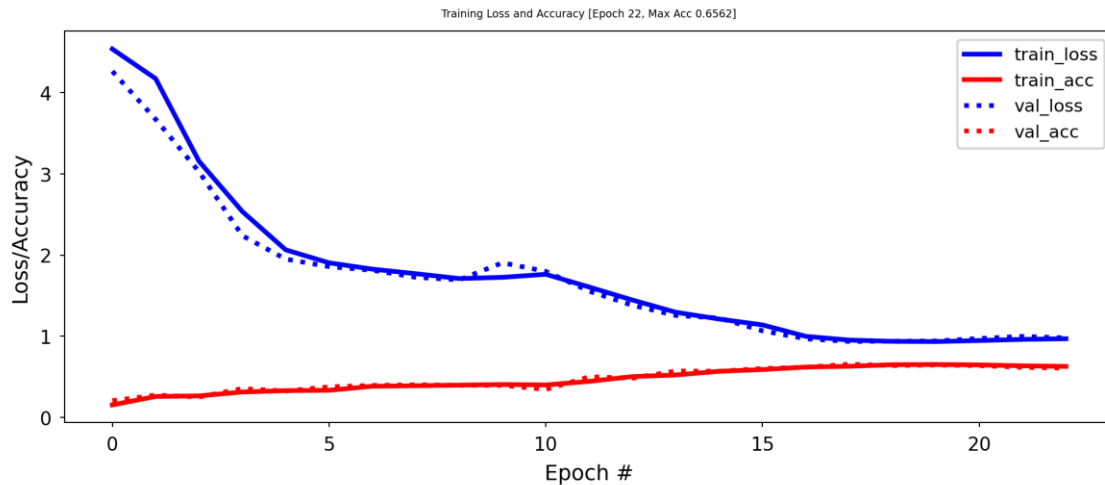
- The number of learnable parameters is called the model *capacity*
- A model with more parameters has a higher *memorisation capacity*
 - Too high capacity causes overfitting, too low causes underfitting
 - In the extreme, the training set can be ‘memorised’ in the weights
- Smaller models are forced it to learn a compressed representation that generalises better
 - Find the sweet spot: e.g. start with few parameters, increase until overfitting stars.



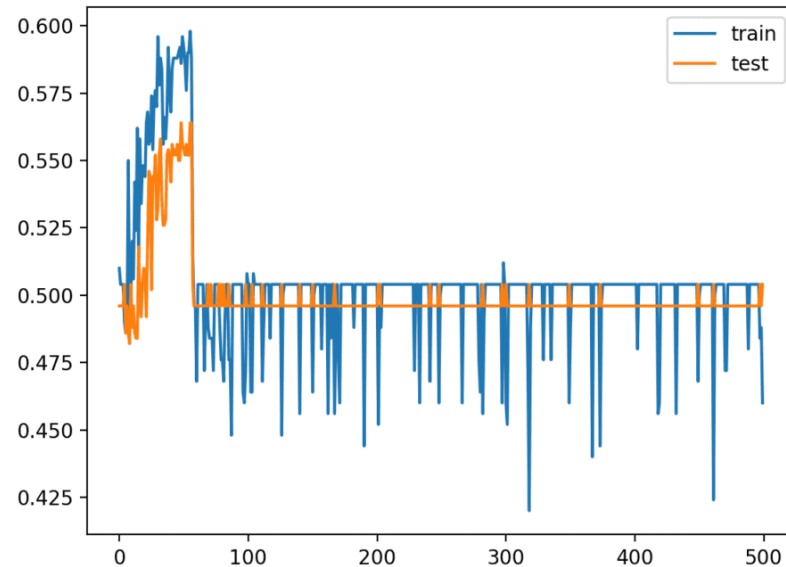
How to know the model is trained well

Information bottleneck

- If a layer is too narrow, it will lose information that can never be recovered by subsequent layers
- *Information bottleneck* theory defines a bound on the capacity of the network
- Imagine that you need to learn 10 outputs (e.g. classes) and your hidden layer has 2 nodes
- Example: bottleneck of 2 nodes, no overfitting, much higher training loss

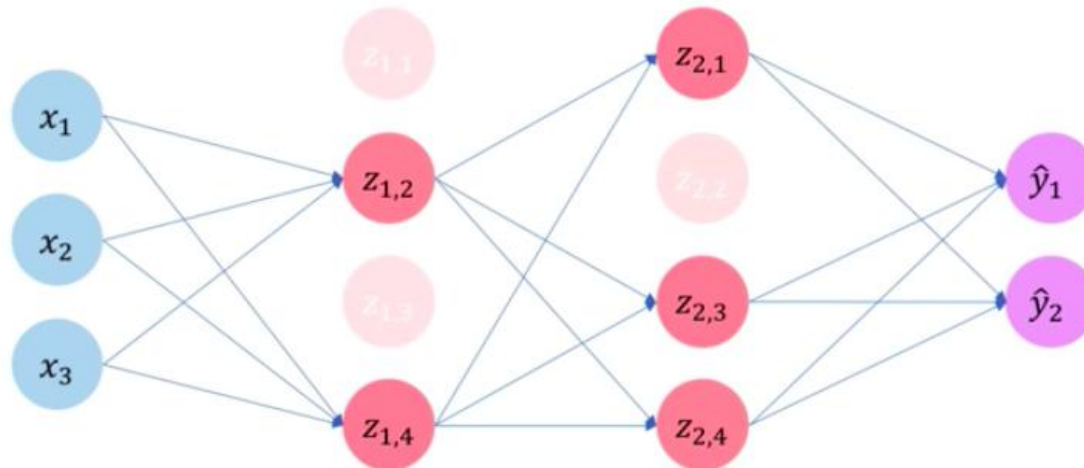


Vanishing gradients



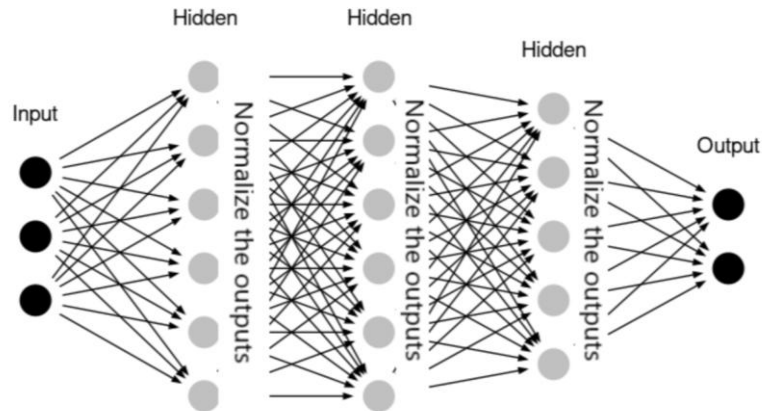
Dropout

- Every iteration, randomly set a number of activations a_i to 0
- *Dropout rate* : fraction of the outputs that are zeroed-out (e.g. 0.1 - 0.5)
- Idea: break up accidental non-significant learned patterns
- At test time, nothing is dropped out, but the output values are scaled down by the dropout rate
 - Balances out that more units are active than during training



Batch Normalisation

- We've seen that scaling the input is important, but what if layer activations become very large?
 - Same problems, starting deeper in the network
- Batch normalisation: normalize the activations of the previous layer within each batch
 - Within a batch, set the mean activation close to 0 and the standard deviation close to 1
 - Across badges, use exponential moving average of batch-wise mean and variance
 - Allows deeper networks less prone to vanishing or exploding gradients



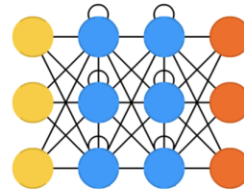
Architectures



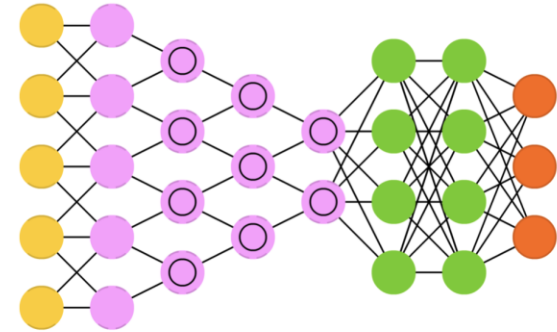
Deep Feed Forward (DFF)



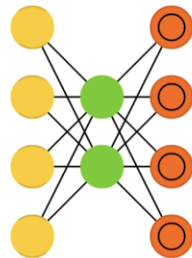
Recurrent Neural Network (RNN)



Deep Convolutional Network (DCN)



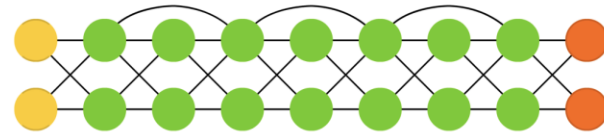
Auto Encoder (AE)



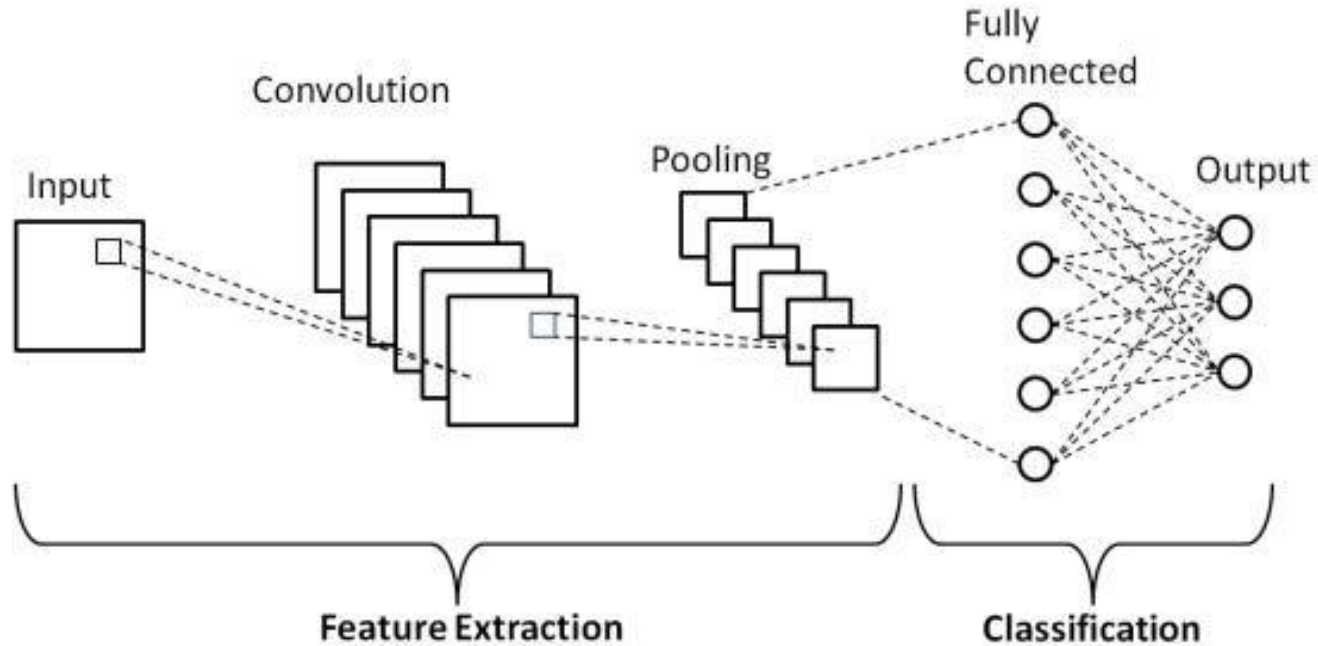
Variational AE (VAE)



Deep Residual Network (DRN)



Convolution



Convolution layer

Input image

9	4	1	2	2
1	1	1	0	4
1	2	1	0	6
1	0	0	2	4
9	6	7	4	1

Filter

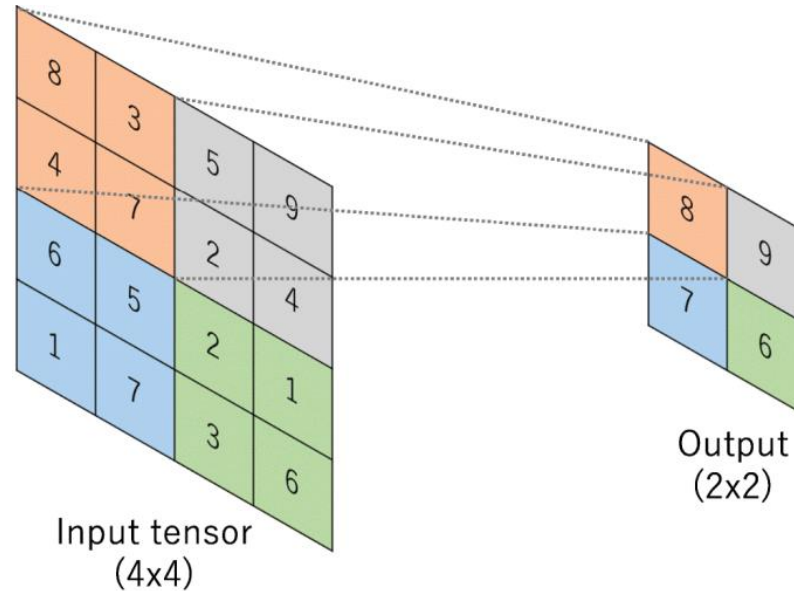
0	2	1
4	1	0
1	0	1

Output array

16		

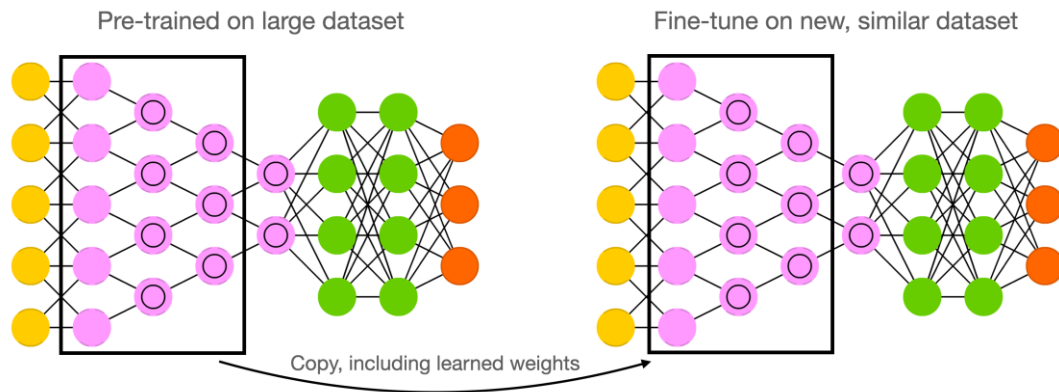
$$\begin{aligned}\text{Output } [0][0] &= (9*0) + (4*2) + (1*4) \\ &+ (1*1) + (1*0) + (1*1) + (2*0) + (1*1) \\ &= 0 + 8 + 1 + 4 + 1 + 0 + 1 + 0 + 1 \\ &= 16\end{aligned}$$

Max pooling



Transfer learning

- Instead of starting from scratch, start from weights previously learned from similar tasks
 - This is, to a big extent, how humans learn so fast
- Transfer learning: learn weights on task T, transfer them to new network
 - Weights can be frozen, or finetuned to the new data
- Only works if the previous task is 'similar' enough
 - Generally, weights learned on very diverse data transfer better
 - Meta-learning: learn a good initialization across many related tasks





Neural Networks in Practice

DeNederlandscheBank

EUROSYSTEM



DataScience
Hub

Content

1. PyTorch as a deep learning framework

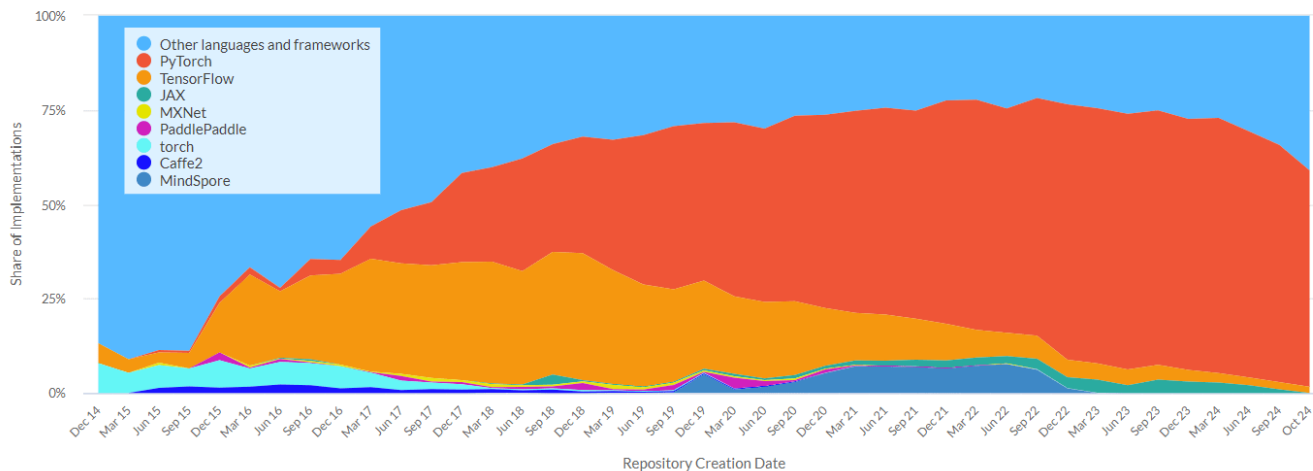
- Tensors, computation graphs, and automatic differentiation

2. Neural networks in PyTorch

- Defining models, loss functions, and optimizers

PyTorch framework

- Originally made by Facebook
- Since Sept 2022 governed by the independent PyTorch foundation
- Focused on GPU calculation
- Automatic differentiation system



RNNs, LSTMs, and Transformers

DeNederlandscheBank

EUROSYSTEEM



DataScience
Hub

Michiel Nijhuis
Iman van Lelyveld

Content

1. Recurrent Neural Networks and LSTMs

- Sequential modelling and longterm dependencies

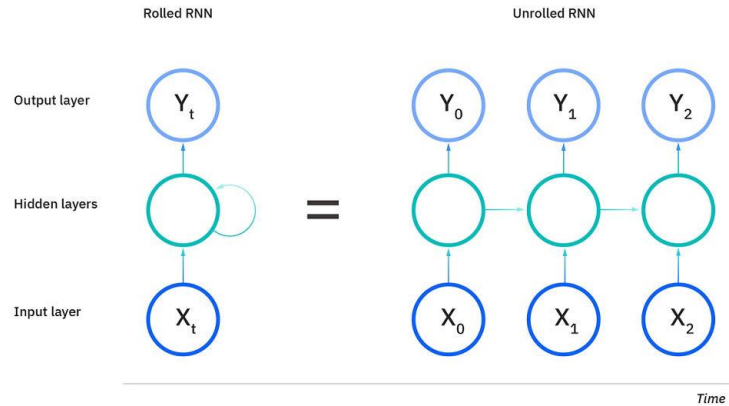
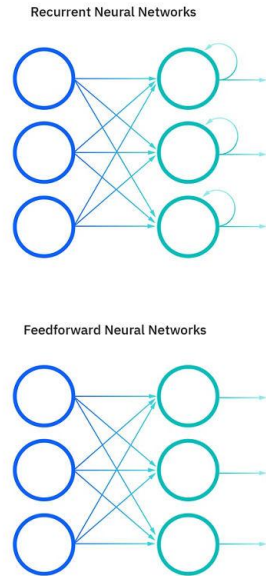
2. Text as input data

- Stand alone Embeddings
- Contextual Embeddings

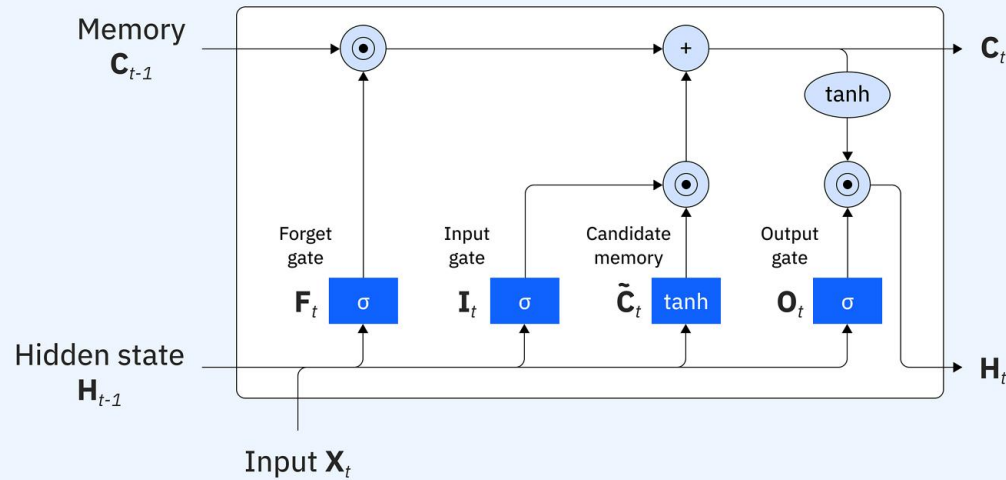
3. Transformers as the foundation of LLMs

- Transformer architecture
- Training a model like ChatGPT

Recurrent neural net



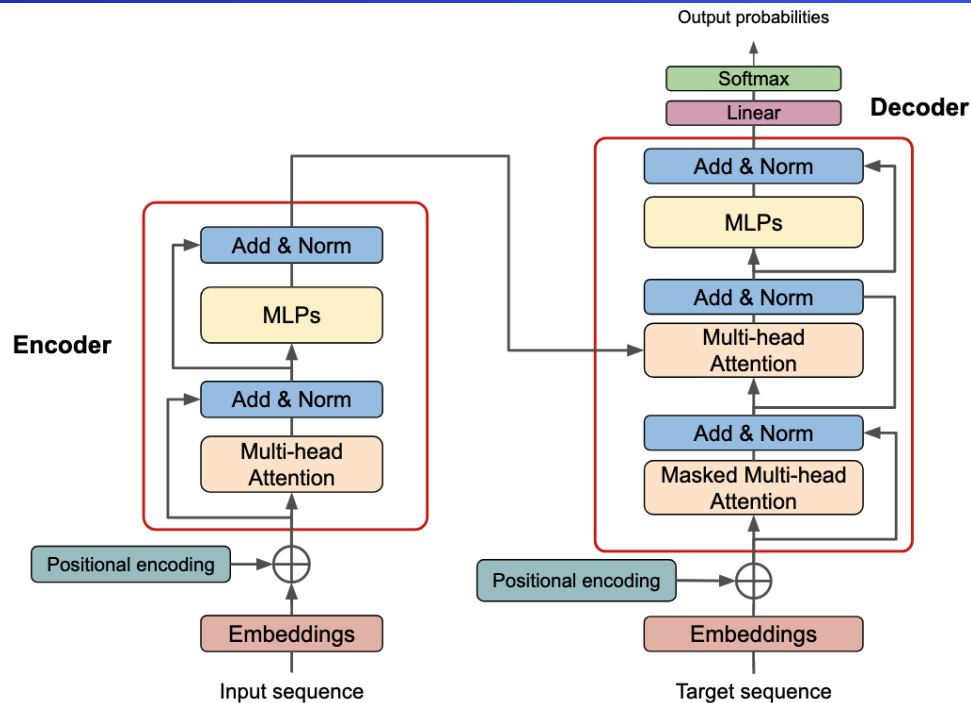
Long short term memory layer



Challenges of Using Text in Neural Networks

- Text is discrete
models must turn words or subwords into numbers before they can learn patterns.
- Meaning depends on context
making the same word hard to interpret across different sentences and domains.
- Language is ambiguous
things like sarcasm, idioms, and implied meaning are difficult to encode.
- Training data can be noisy or biased

Transformer layers



Using text data in a neural network

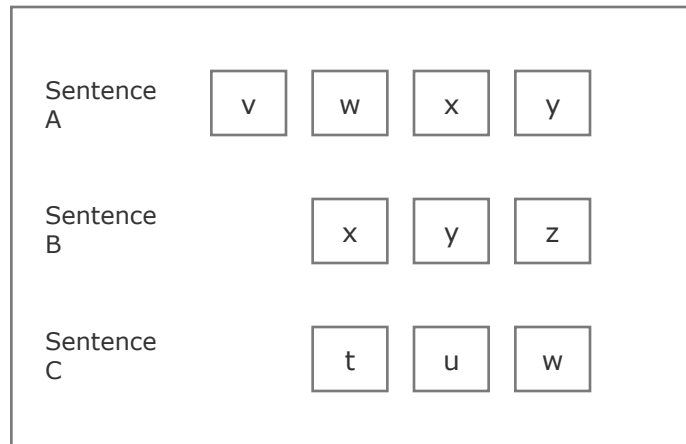


Tokenisation

[GPT token encoder and decoder /
Simon Willison | Observable](#)

Stand alone embeddings

Classify words based on having the same context



Co-occurrence matrix

	v	w	x	y	z	t	u
v	0	1	1	1	0	0	0
w	1	0	1	1	0	1	1
x	1	1	0	2	1	0	0
y	1	1	2	0	1	0	0
z	0	0	1	1	0	0	0
t	0	1	0	0	0	0	1
u	0	1	0	0	0	1	0

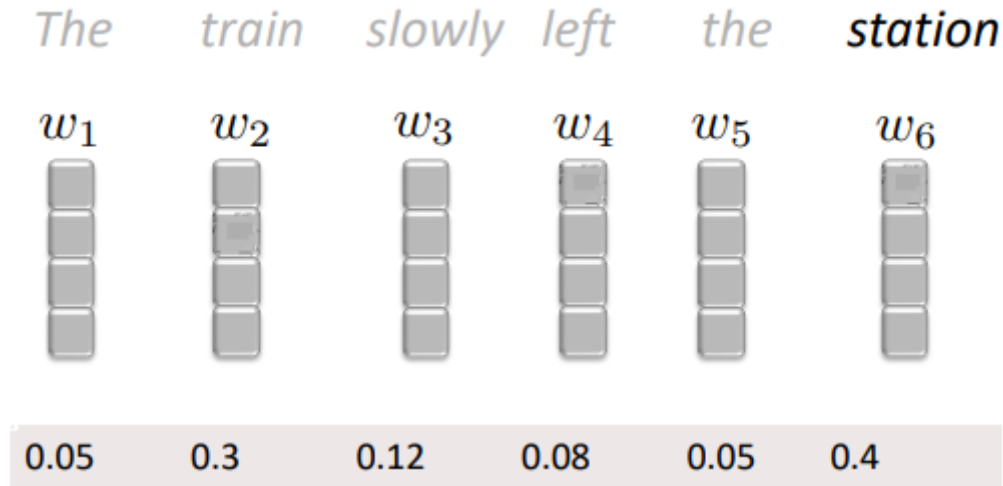
$$X_{i,j} = b_i + b_j + w_i^T w_j$$

Embeddings

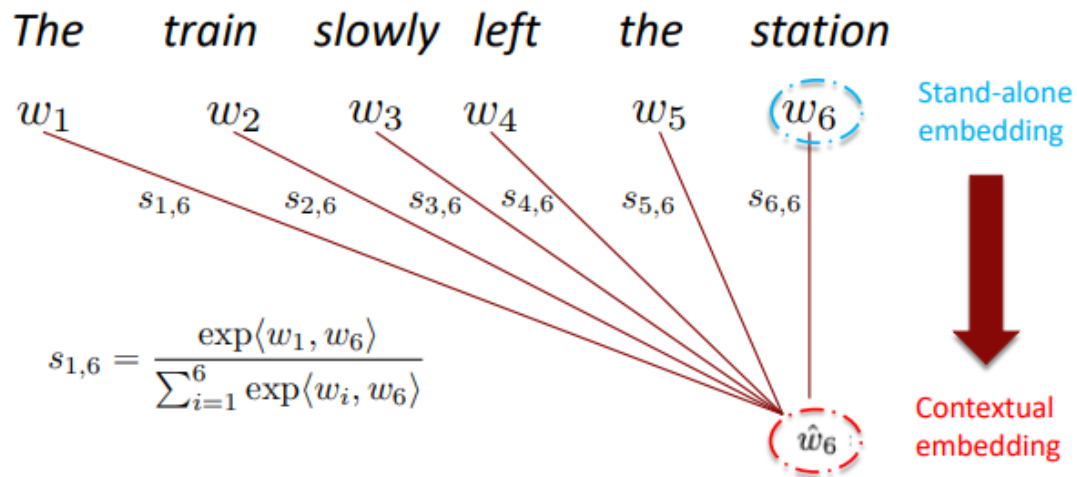
[Embedding projector - visualization of high-dimensional data](#)

Contextual embeddings

Not every word is as import for the context

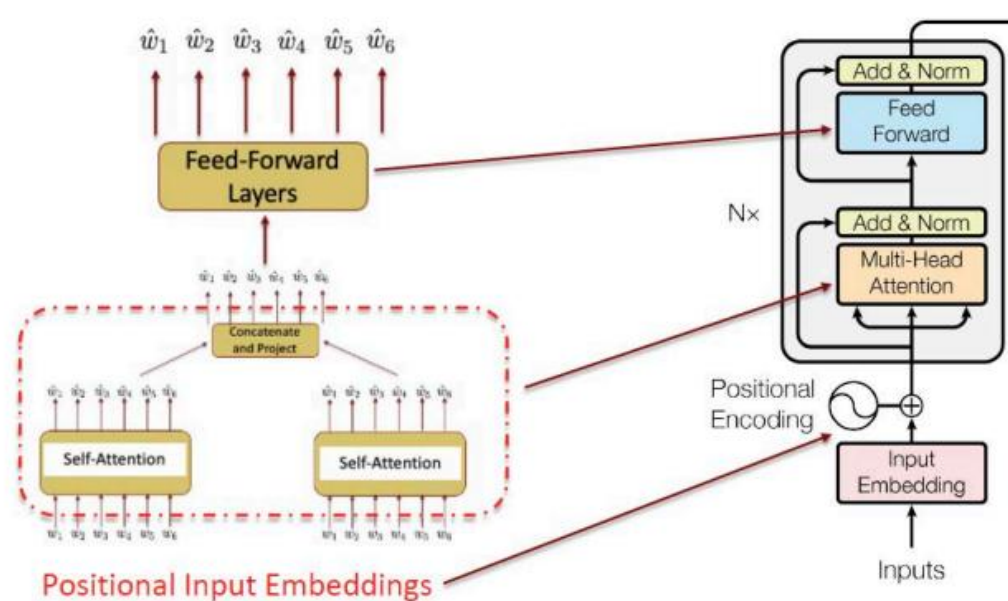


Contextual embeddings

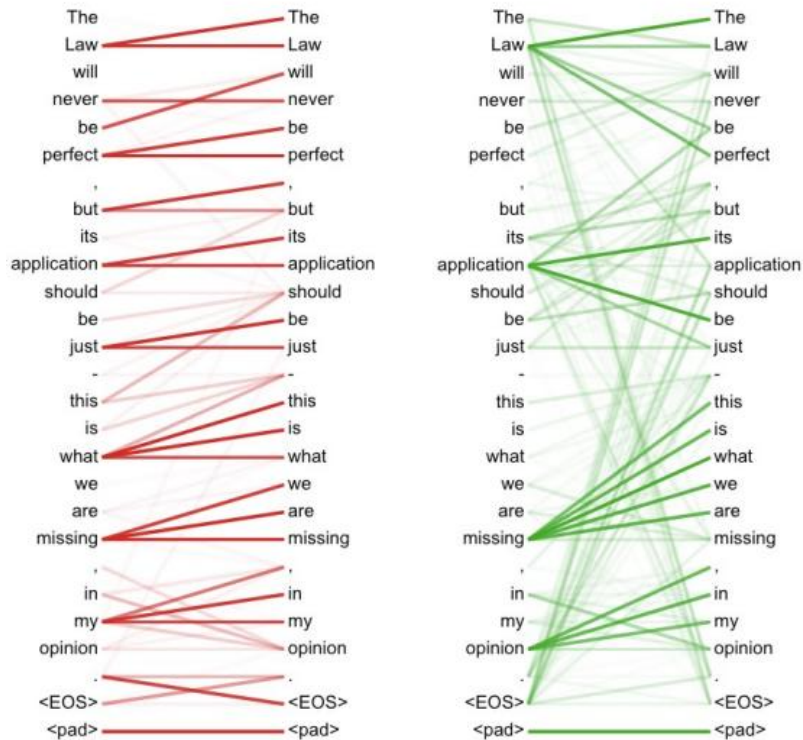


$$\hat{w}_6 = s_{1,6}w_1 + s_{2,6}w_2 + s_{3,6}w_3 + s_{4,6}w_4 + s_{5,6}w_5 + s_{6,6}w_6$$

Adding positional information and a feed-forward layer

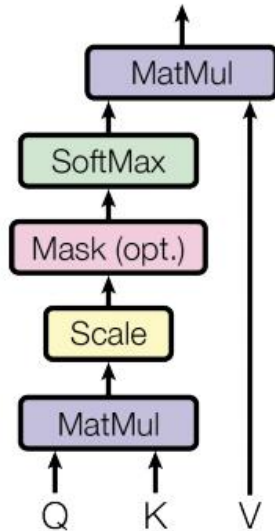


Multiple attention layers

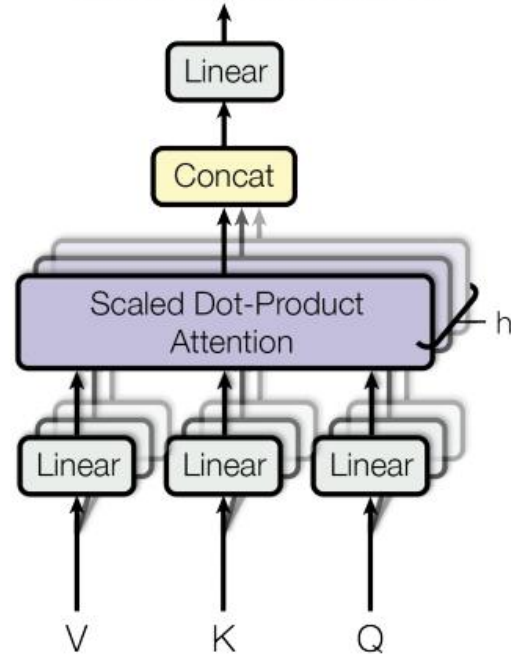


Attention layer

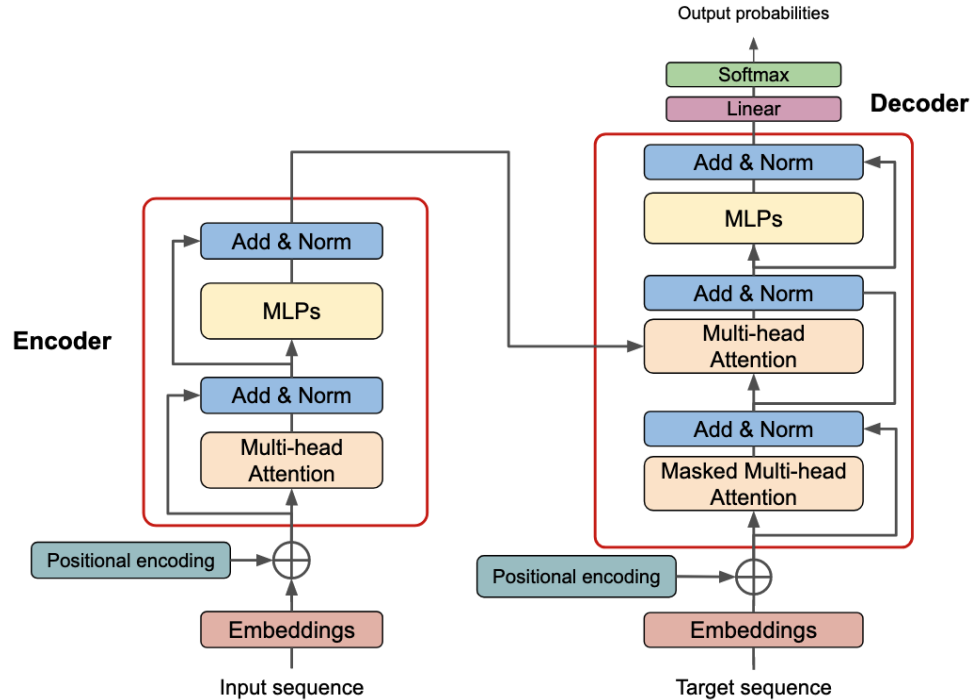
Scaled Dot-Product Attention



Multi-Head Attention



Transformer layers



LLM Pretraining and Fine-Tuning

Pretrain model

- Train on massive tokenized corpora
- Optimize next-token prediction at scale
- Build broad language, facts, and reasoning patterns

Fine-tune

- Adapt the base model to a task or domain
- Use curated examples or feedback data
- Improve tone, quality, and reliability for use